Parallel EM Clustering

HPC4DS
a.y. 2025/2026
University of Trento

Davide Donà - 264467 Andrea Blushi - 264468

July 2026

Code Repository

Contents

1 Introduction

In this work, we address the problem of clustering large datasets using a parallel implementation of the Expectation-Maximization (EM) algorithm for Gaussian Mixture Models (GMMs). Clustering is a fundamental task in unsupervised machine learning, where the goal is to group similar data points together based on their inherent characteristics. The EM algorithm is an iterative procedure for estimating the parameters of GMMs, enabling the identification of latent cluster structures within the data. However, the EM algorithm can be computationally intensive, especially for large datasets, which motivates the need for parallelization to improve efficiency and scalability. In this report, we present two parallel implementations of the EM algorithm for clustering. The first one is based only on MPI, and a second one uses a hybrid approach based on MPI and OpenMP. We evaluate the performance of our implementation under various settings, analyzing its scalability, speedup, and efficiency. Our results demonstrate the effectiveness of parallelization in accelerating the EM algorithm for clustering large datasets.

1.1 Problem Overview

The goal of clustering is to partition a dataset $\mathcal{X} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N\}$ where $\boldsymbol{x}_i \in \mathbb{R}^D$ into K (number of clusters) groups without using label information. Each cluster is represented by a probability distribution, and in our case, they are modeled as Gaussian distributions. The challenge is to estimate the parameters of these distributions–namely the means, covariances, and mixture weights–such that the overall likelihood of the data given the model is maximized.

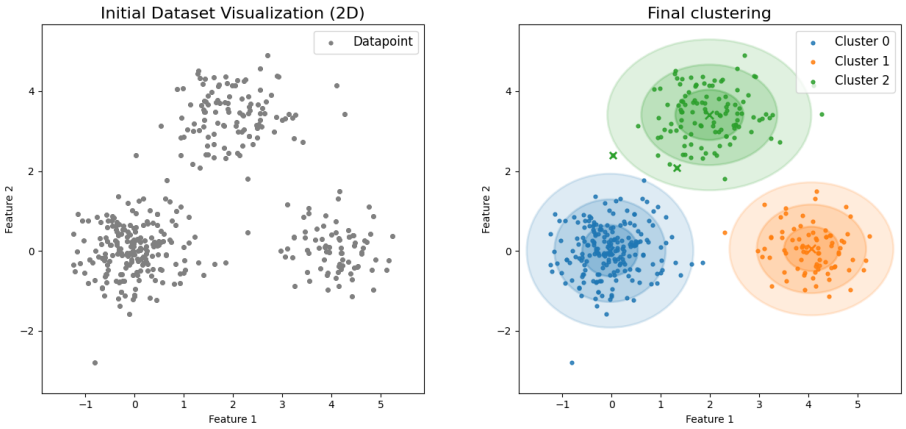

Figure 1: Example of clustering using GMMs with $K = 3$ on a 2D dataset.

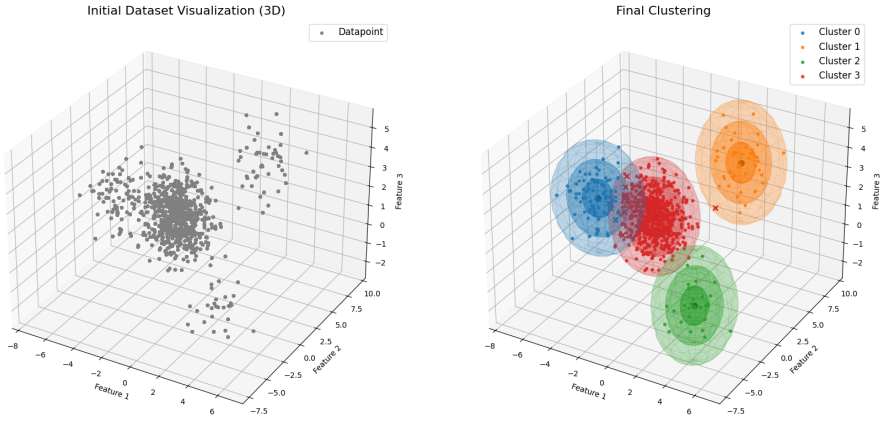

Figure 2: Example of clustering using GMMs with $K = 4$ on a 3D dataset.

1.2 Gaussian Mixture Models

A Gaussian Mixture Model (GMM) [1] is a probabilistic model that represents data as a combination of multiple Gaussian distributions, each with its own mean, covariance, and mixture weight. GMMs are widely used for clustering tasks due to their flexibility in modeling complex data distributions, where data points may naturally cluster around multiple centers with varying shapes and sizes. The goal is to estimate the parameters of the Gaussian components and assign data points to these clusters.
A GMM assumes that each data point is generated from one of K Gaussian components:

$$p(\boldsymbol{x} \mid \theta) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \Sigma_k) \tag{1}$$

where the sum of the mixture weights satisfies the following constraints:

$$\sum_{k=1}^{K} \pi_k = 1, \quad \pi_k \geq 0$$

Here, π_k are the mixture weights, $\boldsymbol{\mu}_k$ are the means, and Σ_k are the covariance matrices of the Gaussian components. The mixture weights π_k represent the prior probability of component k, while the Gaussian density function $\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \Sigma_k)$ models the distribution of data points assigned to that component. The Gaussian density function is defined as:

$$\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \Sigma_k) = \frac{1}{(2\pi)^{D/2}|\Sigma_k|^{1/2}} \exp\left(-\frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_k)^T \Sigma_k^{-1} (\boldsymbol{x} - \boldsymbol{\mu}_k)\right) \tag{2}$$

With this function, we can evaluate the likelihood of a data point $\boldsymbol{x}$ belonging to the k-th Gaussian component.
In our implementation, we adopt a diagonal covariance matrix [2] for each Gaussian component. This assumption implies that the features are uncorrelated within each component, simplifying the covariance matrix to a diagonal form:

$$\Sigma_k = \begin{pmatrix} \sigma_{k,1} & 0 & \cdots & 0 \\ 0 & \sigma_{k,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_{k,D} \end{pmatrix}$$

Then the Gaussian density function simplifies to:

$$\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \Sigma_k) = \frac{1}{(2\pi)^{D/2} \prod_{d=1}^{D} \sigma_{k,d}^{1/2}} \exp\left(-\frac{1}{2} \sum_{d=1}^{D} \frac{(x_d - \mu_{k,d})^2}{\sigma_{k,d}} \right) \tag{3}$$

This simplification reduces computational complexity and memory usage, making it feasible to work with high-dimensional data. In fact, evaluating the Gaussian density in the full-covariance case requires $O(D^3)$ time due to matrix inversion and determinant computation, while the diagonal formulation reduces this to $O(D)$.
At this point, our main problem is to estimate the parameters $\theta = \{\pi_k, \boldsymbol{\mu}_k, \Sigma_k\}_{k=1}^{K}$ of the GMM given the dataset $\mathcal{X}$, as they are unknown in our case.

1.3 Expectation-Maximization algorithm

The Expectation-Maximization (EM) algorithm [3] is an iterative method for finding maximum likelihood estimates of parameters in statistical models, where the model depends on unobserved latent variables. In the context of GMMs, the latent variables represent the cluster assignments of data points. The EM algorithm consists of two main steps: the Expectation step (E-step) and the Maximization step (M-step).

E-Step

Using the current parameter estimates θ, we compute the posterior probabilities, also called responsibilities, that each data point belongs to each Gaussian component. At the first iteration, the parameters are initialized randomly, in order to start the algorithm. The responsibilities γ_{ik}, representing the probability that data point $\boldsymbol{x}_i$ belongs to cluster k, are computed as follows:

$$\gamma_{ik} = p(z_i = k \mid \boldsymbol{x}_i, \theta) = \frac{\pi_k \mathcal{N}(\boldsymbol{x}_i \mid \boldsymbol{\mu}_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(\boldsymbol{x}_i \mid \boldsymbol{\mu}_j, \Sigma_j)} \tag{4}$$

M-Step

Using the responsibilities γ_{ik} computed in the E-step, we update the parameters of the GMM to maximize the expected complete-data log-likelihood. The updates for each component $k = 1, \ldots, K$ are as follows:

$$N_k = \sum_{i=1}^{N} \gamma_{ik} \tag{5}$$

Update of means:

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{i=1}^{N} \gamma_{ik} \boldsymbol{x}_i$$

Update of covariance matrices:

$$\Sigma_k = \frac{1}{N_k} \sum_{i=1}^{N} \gamma_{ik} (\boldsymbol{x}_i - \boldsymbol{\mu}_k)(\boldsymbol{x}_i - \boldsymbol{\mu}_k)^T$$

Update of mixture weights:

$$\pi_k = \frac{N_k}{N}$$

After updating the parameters, the E-step and M-step are repeated until convergence. Convergence is typically assessed by monitoring the change in log-likelihood between successive iterations. The process is considered converged if the change is below a predefined threshold ϵ. Alternatively, a maximum number of iterations can be set to prevent infinite loops.
The log-likelihood of the data given the model parameters is computed as:

$$\mathcal{L}(\theta) = \sum_{i=1}^{N} \log \left(\sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{x}_i \mid \boldsymbol{\mu}_k, \Sigma_k) \right) \tag{6}$$

Clustering Assignment

In order to cluster the dataset [4] using the EM algorithm for GMMs, we simply assign each data point to the cluster corresponding to the Gaussian component with the highest responsibility. This is only done after the algorithm has converged or reached the maximum number of iterations.

$$\hat{z}_i = \arg\max_k p(z_i = k \mid \boldsymbol{x}_i, \theta) = \arg\max_k \gamma_{ik} \tag{7}$$

Pseudocode

In the following, we provide pseudocode for the overall EM algorithm for GMMs. More in depth pseudocode for each step is provided in the Appendix A.

Algorithm 1 Expectation-Maximization for GMM clustering

```
Initialize parameters {π_k, μ_k, Σ_k}_{k=1}^K
for i = 1 ... MAX_ITER do
    E-step: compute responsibilities using (4)
    M-step: update parameters using (5)
    Compute log-likelihood L according to (6)
    if Converged then
        break
    end if
end for
Clustering: assign labels using (7)
```

1.4 Computational Complexity analysis

The computational cost of the EM algorithm is determined by the number of data points N, clusters K, and feature dimensions D. Both the E-step and M-step require $O(NKD)$ operations per iteration, resulting in an overall per-iteration complexity of $O(NKD)$. For large datasets, this can become a significant bottleneck, motivating the need for parallelization to improve its efficiency.

2 Parallel Design

Current state-of-the-art parallelization techniques for the EM algorithm primarily focus on data parallelism approaches, where the dataset is partitioned across multiple processing units so that computations can be performed simultaneously on different subsets of the data. [5] presents a GPU-based implementation of the EM algorithm that leverages the massive parallelism of GPUs to accelerate the computation of both the E-step and M-step using an Asynchronous Model Update strategy. Similarly, [6] proposes a simple multithreaded version of the EM algorithm for finite mixture models.
Both approaches demonstrate significant speedups compared to traditional sequential implementations, making them suitable for large-scale data analysis tasks.

2.1 Our Approach

Building on the insights of the reviewed literature, our approach adopts a data-parallelism strategy to enhance the performance of the EM algorithm for GMMs. The data samples are evenly distributed across multiple processing units to compute local statistics, which are then aggregated to update the global model parameters. This design effectively reduces the computational load on each processing unit and minimizes the overall execution time.

E-step Parallelization

The E-step is trivially parallelizable, as responsibilities for each data point are computed independently of each other. Each processing unit can execute the standard E-step on its local data chunk, storing the results for the subsequent M-step aggregation.

M-step Parallelization

Differently, the M-step requires a more coordinated approach to ensure globally consistent parameter updates. Each processing unit computes local accumulators for the sufficient statistics (i.e., effective counts, weighted sums for the means, and weighted squared differences for the covariances) based on its local responsibilities. These local results are then aggregated, in order to finalize the global parameter updates.

Clustering Assignment

The clustering assignment step can be parallelized in a similar manner to the E-step. Each processing unit determines the cluster assignments for its local data chunk using the final responsibilities from the preceding E-step. Since the assignments are independent for each data point, this step requires no inter-process communication. Once all units complete their local clustering assignments, the results can be gathered if needed for further analysis or output.

Performance Considerations

Theoretically, this design reduces the computation times associated with the dataset size. Based on the computational complexity analysis in Section 1.4, ideally, the overall time complexity of the EM algorithm is reduced from $O(NKD)$ to $O\left(\frac{NKD}{P}\right)$, where P is the number of processing units.
We assume that the number of clusters K and dimensionality D are significantly smaller than the number of data points N, which is a common characteristic of many real-world large-scale clustering applications. Under this assumption, we can consider K and D as multiplicative constants. The computational cost is dominated by N, making our parallelization strategy particularly effective: distributing the data across processing units yields near-linear reductions in execution time. However, as the number of processing units P increases, the benefits of parallelization can diminish due to the communication overhead required for the M-step, which can outweigh the computational gains. This limits the scalability of the approach and creates a bottleneck. Nevertheless, the M-step itself remains manageable, as the size of the parameters that need to be communicated depends only on K and D.

2.2 Hybrid Parallelism

A hybrid parallelization strategy could, in principle, mitigate some of the problems observed in pure process-based data-parallel design. Since the size of the data exchanged during inter-process communication depends only on K and D, reducing the number of processes while increasing the number of threads per process reduces the number of communication events, thus lowering the overall communication overhead. In such a configuration, each process operates on a larger portion of the dataset, allowing more computation to occur within a shared-memory environment, where threads can efficiently cooperate without the need for expensive message passing. As a result, both the frequency and volume of inter-process communication taking place during the M-step can be significantly reduced.

2.2.1 Data dependencies

To assess the feasibility and potential benefits of hybrid parallelism within our implementation, we first examined the data dependencies inherent to the EM algorithm.

E-step

```
1  double e_step(double *X, int N, Metadata *metadata, ClusterParams *cluster_params, double *gamma){
2      double log_likelihood = 0.0;
3      for(int i = 0; i < N; i++) {
4          double denom = 0.0;
5          double *x = &X[i*metadata->D];
6          for (int k = 0; k < metadata->K; k++) {
7              double *mu_k = &cluster_params->mu[k*metadata->D];
8              double *sigma_k = &cluster_params->sigma[k*metadata->D];
9              gamma[i*metadata->K + k] = cluster_params->pi[k] * gaussian_multi_diag(x, mu_k, sigma_k, metadata->D);
10             denom += gamma[i*metadata->K + k];
11         }
12         for (int k = 0; k < metadata->K; k++) gamma[i*metadata->K + k] /= denom;
13         log_likelihood += log(denom);
14     }
15     return log_likelihood;
16 }
```

Figure 3: E-step implementation code snippet.

ID	Memory Location	Earlier Statement			Later Statement			Loop-carried?	Kind of Dataflow
		Line	Iteration	Access	Line	Iteration	Access		
1	`log_likelihood`	13	i	write	13	$i+1$	read	yes	flow
2	`denom`	10	i,k	write	10	$i,k+1$	read	yes	flow
3	`gamma`$(i*k+k)$	9	i,k	write	10	i,k	read	no	flow
4	`gamma`$(i*k+k)$	10	i,k	read	12	i,k	write	no	anti
5	`denom`	10	i,k	write	12	i,k	read	no	flow
6	`denom`	4	i	write	10	i	read	no	flow

The only true loop-carried dependency is related to the accumulation of the log-likelihood over all data points (ID 1). To address this, we need to implement a reduction operation to safely combine the partial log-likelihoods computed

by each thread. The dependencies that are not loop-carried (IDs 3-6) do not pose a problem for parallelization. Since the parallel decomposition ensures that each thread operates on different iterations of the loop, these dependencies are naturally handled without synchronization. Also, dependency (ID 2) can be considered non-problematic. Since it is a dependency carried within the inner loop over K, and the inner loop is executed sequentially by each thread, no inter-thread synchronization is required.

M-step

In this section, we analyze only the true data dependencies found within the M-step implementation. It's important to note that there is a flow dependency (Read-After-Write) involving the `gamma` array that spans between the E-step and M-step. However, since the E-step must be fully completed before the M-step begins, this dependency does not impact the parallelization of the M-step itself.

```
1  void m_step( double *X, Metadata *metadata, ClusterParams *cluster_params, Accumulators *acc, double *gamma){
2      reset_accumulators(acc, metadata);
3      for (int i = 0; i < metadata->N; i++) {
4          double *x = &X[i*metadata->D];
5          for (int k = 0; k < metadata->K; k++) {
6              acc->N_k[k] += gamma[i*metadata->K + k];
7              for (int d = 0; d < metadata->D; d++) {
8                  acc->mu_k[k*metadata->D + d] += gamma[i*metadata->K + k] * x[d];
9              }
10         }
11     }
12     for (int k = 0; k<metadata->K; k++) {
13         for( int d = 0; d < metadata->D; d++) {
14             cluster_params->mu[k*metadata->D + d] = acc->mu_k[k*metadata->D +d] / acc->N_k[k];
15         }
16     }
17     for (int i = 0; i < metadata->N; i++) {
18         double *x = &X[i * metadata->D];
19         for (int k = 0; k < metadata->K; k++) {
20             for (int d = 0; d < metadata->D; d++) {
21                 double diff = x[d] - cluster_params->mu[k * metadata->D + d];
22                 acc->sigma_k[k * metadata->D + d] += gamma[i * metadata->K + k] * diff * diff;
23             }
24         }
25     }
26     for (int k = 0; k < metadata->K; k++) {
27         for (int d = 0; d < metadata->D; d++) {
28             cluster_params->sigma[k * metadata->D + d] = acc->sigma_k[k * metadata->D + d] / acc->N_k[k];
29         }
30         cluster_params->pi[k] = acc->N_k[k] / (double)metadata->N;
31     }
32 }
```

Figure 4: M-step implementation code snippet.

ID	Memory Location	Earlier Statement			Later Statement			Loop-carried?	Kind of Dataflow
		Line	Iteration	Access	Line	Iteration	Access		
1	`N_k`(k)	6	i,k	write	6	$i+1,k$	read	yes	flow
2	`mu_k`$(k*D+d)$	8	i,k,d	write	8	$i+1,k,d$	read	yes	flow
3	`N_k`(k)	6	i,k	write	14	k	read	no	flow
4	`mu_k`$(k*D+d)$	8	i,k,d	write	14	k,d	read	no	flow
5	`mu`$(k*D+d)$	14	k,d	write	21	k,d	read	no	flow
6	`sigma_k`$(k*D+d)$	22	i,k,d	write	22	$i+1,k,d$	read	yes	flow
7	`sigma_k`$(k*D+d)$	22	k,d	write	28	k,d	read	no	flow

The loop-carried dependencies (IDs 1, 2, and 6) are true dependencies that need to be addressed. These can be resolved by implementing reduction operations for the accumulators `N_k`, `mu_k`, and `sigma_k`, allowing each thread to compute partial sums that are then safely combined. The other dependencies (IDs 3, 4, 5, and 7) are not loop-carried and occur after the reduction operations, meaning they do not interfere with the parallel computation within the loop. It's important to note that, before proceeding with the E-step of the next iteration, we must ensure that all threads have the same parameters (`mu`, `sigma`, and `pi`). This can be resolved by first having a reduction on `sigma_k`, then all global parameters can be updated by all threads independently.

2.3 Different parallelization strategies

Several alternative parallelization strategies were considered but ultimately not implemented in this project, as they are less well suited to the structure of the EM algorithm than data parallelism.

- *Task parallelization*: This approach was deemed unsuitable due to the inherent characteristics of the EM algorithm. Each task would require access to the entire dataset in order to compute the responsibilities and update the parameters, resulting in substantial communication overhead, effectively negating any potential performance gains. Moreover, the iterative nature of the EM algorithm would require frequent synchronization between tasks, further increasing the overhead.

- *Pipeline parallelization*: This strategy was not applicable. The algorithm does not follow an independent sequence of stages that can be overlapped in a pipeline fashion; instead, it is inherently iterative. Iteration $t + 1$ cannot begin until step t is fully complete, preventing effective pipeline overlap.

- *Model parallelization*: This approach involves splitting the set of Gaussian components across devices when K is large; each unit holds a subset of parameters and participates in responsibility computation via broadcast/gather. While theoretically feasible, this strategy introduces significant communication complexity, particularly during the E-step, where each device must compute responsibilities for all data points using parameters distributed across all other devices. As a result, this approach is only viable if K is massive and N is small, which contradicts the typical use case for GMMs.

Based on this analysis and the insights gained from the state-of-the-art literature, we conclude that other parallelization strategies besides data-parallelism are not suitable for efficient implementation of the EM algorithm.

3 Implementation

The algorithm was implemented in C, inspired by the Python codebase provided in [7]. The parallelization was achieved using MPI to enable distributed computing across multiple processors, while hybrid parallelization was achieved by integrating OpenMP to exploit multi-threading within each process. Additionally, a suite of Python scripts, Bash scripts and Jupyter Notebooks were developed to support systematic validation and analysis of the performance of our implementation.

Data generation

Synthetic datasets were generated in Python to produce GMM-distributed data points. This approach allowed for precise control over key parameters, including the number of samples, dimensionality, and cluster count, as well as the means and variances of each Gaussian component. This controlled environment was essential for rigorously testing and validating the correctness and performance of our parallel EM implementation.
It is worth noting that the data generator produces clusters with diagonal covariance matrices, meaning that features within each cluster are uncorrelated. The implementation also supports both balanced (equal number of samples per cluster) and unbalanced cluster sizes.
To evaluate clustering accuracy, true cluster labels are generated alongside the data points. For each dataset, an additional metadata file is created, containing the relevant generation parameters such as the number of samples, dimensionality, cluster weights, means, and variances. This metadata simplifies the initialization of the EM algorithm by providing all necessary information without requiring manual configuration.

Accuracy Checks

To ensure the correctness of the implementation, particularly during the initial testing phase, we performed extensive validation by comparing the outputs of the algorithm against the ground truth labels provided by the data generator. Clustering performance was evaluated using permutation-invariant accuracy metrics (using the Python library `scikit-learn`). This approach accounts for the arbitrary labeling inherent in unsupervised learning. By resolving label permutations, we could accurately measure how well the algorithm clustered the data points according to their true underlying distributions.

Gaussian Density Computation

The Gaussian density function was implemented as a diagonal multivariate Gaussian, consistent with the data generation process. Crucially, this simplification does not compromise the validity of the parallel performance analysis.

Since the density function must still be computed for each data point and cluster independently, the parallelization remains effective no matter the covariance structure.

3.1 Parallel Algorithm Implementation

This section details the implementation of the parallel EM algorithm using MPI. The pseudocode for both the E-step and M-step is presented, highlighting the specific MPI communication directives and synchronization points employed. The complete source code is available in the project repository.

Data distribution

The dataset, together with its metadata, is initially loaded by the root process (rank 0). First, the metadata (number of samples, dimensions, and number of clusters) is broadcast to all processes using `MPI_Bcast`. Based on this information, each process computes the size of its local workload.
The dataset is then partitioned and distributed via `MPI_Scatterv`. The vector variant (`v`) is employed here to handle cases where the number of samples N is not perfectly divisible by the number of processes P, allowing for uneven chunk sizes. After receiving their respective data partition X_{loc}, all processes proceed with the computation in parallel.
At the end of the execution (specifically after the clustering assignment step), each process holds the predicted labels for its local data chunk. These local predicted labels are gathered back to the root process using `MPI_Gatherv`, in order to be written back to a file for accuracy evaluation.

E-step Implementation

The E-step is inherently data-parallel and does not require inter-process communication during execution. As a result, only minimal modifications to the sequential algorithm (Algorithm 4) were necessary. Each process computes the responsibilities for its local data chunk independently, using the current global model parameters.
A key optimization in our implementation is the on-the-fly computation of the log-likelihood during the responsibility normalization loop. This avoids a redundant pass over the data, improving cache efficiency.

Algorithm 2 E-step: Computation of Responsibilities

Require: Local data chunk $X_{\text{loc}} \in \mathbb{R}^{N_{\text{loc}} \times D}$, local parameters $\mu_k^{\text{loc}}, \sigma_k^{\text{loc}}, \pi_k^{\text{loc}}$ for $k = 1, \ldots, K$
Ensure: Local responsibilities γ_{ik}^{loc}

for $i = 1$ to N_{loc} **do**
  denom $\leftarrow 0$
  for $k = 1$ to K **do**
    $\gamma_{ik}^{\text{loc}} \leftarrow \pi_k^{\text{loc}} \cdot \mathcal{N}(X_{\text{loc}}[i] \mid \mu_k^{\text{loc}}, \Sigma_k^{\text{loc}})$ ▷ Gaussian density using 6
    denom $\leftarrow$ denom $+ \gamma_{ik}$
  end for
  for $k = 1$ to K **do**
    $\gamma_{ik}^{\text{loc}} \leftarrow \gamma_{ik}^{\text{loc}}$/denom
  end for
end for

M-step Implementation

The M-step is implemented as outlined in Algorithm 3. Each process first initializes local accumulators for the sufficient statistics and computes their partial distribution. Upon completion, the local results are aggregated across all processes via `MPI_Allreduce`, which was preferred over `MPI_Reduce` followed by a broadcast, since the combined results were needed by all processes for the subsequent steps. Finally, each process updates the model parameters using the aggregated statistics.

Algorithm 3 Parallel M-step

Require: Local data chunk $X_{\text{loc}} \in \mathbb{R}^{N_{\text{loc}} \times D}$, local responsibilities γ_{ik}^{loc}
Ensure: Updated parameters μ_k, σ_k, π_k

Initialize local accumulators:
$N_k^{\text{loc}} \leftarrow 0, \quad \mu_{kd}^{\text{loc}} \leftarrow 0, \quad \sigma_{kd}^{\text{loc}} \leftarrow 0$

Local accumulation of N_k^{loc} and weighted means μ_{kd}^{loc}
for $i = 1$ to N_{loc} **do**
  for $k = 1$ to K **do**
    $N_k^{\text{loc}} \leftarrow N_k^{\text{loc}} + \gamma_{ik}^{\text{loc}}$
    for $d = 1$ to D **do**
      $\mu_{kd}^{\text{loc}} \leftarrow \mu_{kd}^{\text{loc}} + \gamma_{ik}^{\text{loc}} \, X_{\text{loc}}[i, d]$
    end for
  end for
end for
AllReduce for N_k and mean accumulators μ_{kd}^{acc}

Finalize means μ_{kd}
for $k = 1$ to K **do**
  if $N_k \leq 0$ **then**
    $N_k \leftarrow \varepsilon$
  end if
  for $d = 1$ to D **do**
    $\mu_{kd} \leftarrow \mu_{kd}^{acc} / N_k$
  end for
end for

Local accumulation of variance σ_{kd}^{loc} numerators
for $i = 1$ to N_{loc} **do**
  for $k = 1$ to K **do**
    for $d = 1$ to D **do**
      $\sigma_{kd}^{\text{loc}} \leftarrow \sigma_{kd}^{\text{loc}} + \gamma_{ik}^{\text{loc}} \cdot (X_{\text{loc}}[i, d] - \mu_{kd})^2$
    end for
  end for
end for
AllReduce for variance accumulators σ_{kd}^{acc}

Finalize variances and mixture weights σ_{kd}, π_k
for $k = 1$ to K **do**
  for $d = 1$ to D **do**
    $\sigma_{kd} \leftarrow \sigma_{kd}^{acc} / N_k$
  end for
  $\pi_k \leftarrow N_k / N$
end for

3.2 Hybrid Parallel Algorithm Implementation

A hybrid parallel implementation was developed by integrating OpenMP directives into the MPI-based code. Each MPI process spawns multiple threads, reducing the communication overhead and utilizing shared memory. We now focus on the specific implementation details of both the E-step and M-step, but we also parallelized the clustering assignment step in a similar manner to the E-step.

E-step Implementation

The E-step was parallelized using the `#pragma omp parallel for` directive to distribute the computation of the responsibilities across multiple threads. A `reduction` clause was required to safely accumulate the log-likelihood, ensuring correctness and avoiding data races during parallel execution. A static scheduling strategy was chosen, as the computational workload per data point is uniform, resulting in good load balance and minimal scheduling overhead.
Regarding the Gaussian density computation, we wanted to use the `#pragma simd` directive to enable vectorization. Theoretically, it would have improved performance by allowing the compiler to generate SIMD instructions for parallel processing of data. However, due to incompatibilities with the compiler used by the HPC cluster, we were unable to successfully apply it.

M-step Implementation

In the M-step, OpenMP directives were used to parallelize the local accumulation of sufficient statistics. The `#pragma omp parallel` directive defines a parallel region, with a static scheduling strategy similar to the E-step. Each thread maintains private accumulators for the cluster counts, weighted means, and variances to avoid data races. After completing the parallel accumulation loops, `#pragma omp barrier` directives are used to synchronize threads before aggregating into the local accumulators.
Because the HPC cluster compiler did not support the vector reduction clause, thread-private accumulators were manually implemented for each sufficient statistic. During the aggregation of N_k and μ_{kd}, the `#pragma omp nowait` directive is applied to allow overlapping computation and communication, since these operations were independent. Once local aggregation is complete, the M-step proceeds with the `MPI_Allreduce` operation to combine results across all MPI processes, followed by the final parameter updates.
We also experimented with the `#pragma omp collapse` directive to parallelize nested loops over clusters and features. However, benchmarking showed negligible performance improvement. This is likely due to the overhead of managing the collapsed loop iterations combined with the relatively small and uneven workload per iteration, which limits the potential benefits of loop collapsing.

4 Performance and Scalability Analysis

To evaluate the performance and scalability of our parallel EM clustering implementation, we conducted a series of experiments, focusing on different dataset sizes and varying the number of processes. Each configuration was executed multiple times to ensure the reliability of the results, we report only the average execution times. We ignored the initial data loading and scattering phases, as they are not part of the core EM algorithm and would skew the performance measurements. Also, this would be easily resolvable in a real-world scenario by using a distributed file system or parallel I/O techniques.
To ensure a fair comparison, all the experiments were conducted by ignoring the convergence criterion, thus running a fixed number of 100 EM iterations for each configuration.
For our benchmarking experiments, we used a `pack:excl` setting on the cluster to have exclusive access to the allocated nodes and try to allocate all processes on the same node.

4.1 Our Approach Results

In this analysis, we keep the number of features ($D = 50$) and number of clusters ($K = 15$) constant, while varying the number of data points (N) and the number of processes (P).

N_Processes	**0.156M**	**0.312M**	**0.625M**	**1.25M**	**2.5M**	**5M**	**10M**
1	281.444	563.894	1027.496	2116.075	4553.415	8741.255	18373.166
2	140.678	261.653	572.785	1013.512	2058.963	4206.922	8887.233
4	71.339	132.982	283.020	522.856	1054.098	2112.717	4437.179
8	34.587	67.079	146.560	280.315	510.900	1100.518	2319.227
16	19.760	34.806	69.208	139.989	264.852	582.296	1153.750
32	9.425	18.796	40.770	75.046	146.378	290.347	601.705
64	5.164	10.042	20.823	41.247	77.118	150.703	321.383

Table 1: Execution times (in seconds) for different dataset sizes and number of processes.

To better understand the scalability of our implementation, we computed the speedup and efficiency for each configuration.

N_Processes	0.156M	0.312M	0.625M	1.25M	2.5M	5M	10M
1	1.000	1.000	1.000	1.000	1.000	1.000	1.000
2	2.001	2.155	1.794	2.088	2.212	2.078	2.067
4	3.945	4.240	3.630	4.047	4.320	4.137	4.141
8	8.137	8.406	7.011	7.549	8.913	7.943	7.922
16	14.243	16.201	14.846	15.116	17.192	15.012	15.925
32	29.862	30.001	25.202	28.197	31.107	30.106	30.535
64	54.505	56.154	49.343	51.303	59.045	58.003	57.169

Table 2: Speedup (T_{serial}/T_p) for different dataset sizes and number of processes p.

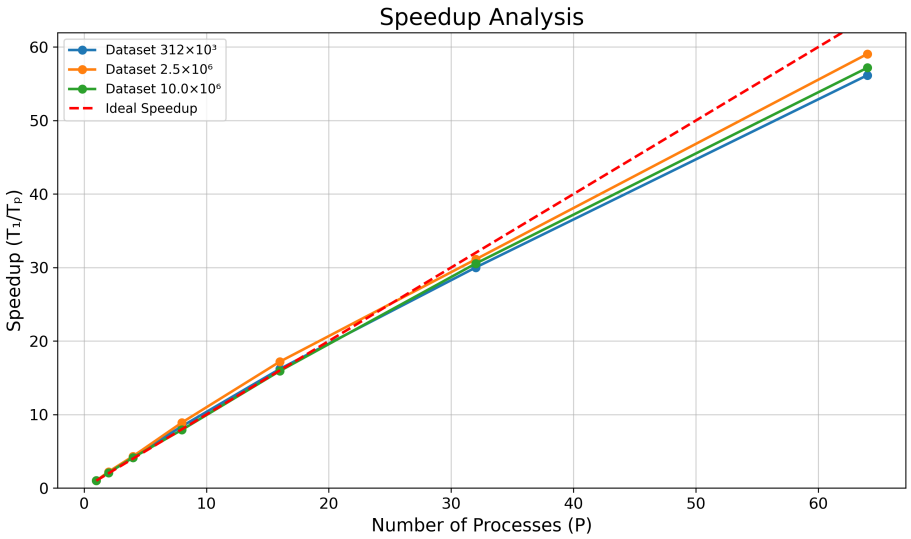

Figure 5: Speedup plot for different dataset sizes (small, medium, large). The dashed line represents the ideal linear speedup.

N_Processes	0.156M	0.312M	0.625M	1.25M	2.5M	5M	10M
1	1.000	1.000	1.000	1.000	1.000	1.000	1.000
2	1.000	1.078	0.897	1.044	1.106	1.039	1.034
4	0.986	1.060	0.908	1.012	1.080	1.034	1.035
8	1.017	1.051	0.876	0.944	1.114	0.993	0.990
16	0.890	1.013	0.928	0.945	1.075	0.938	0.995
32	0.933	0.938	0.788	0.881	0.972	0.941	0.954
64	0.852	0.877	0.771	0.802	0.923	0.906	0.893

Table 3: Efficiency (Speedup / p) for different dataset sizes and number of processes.

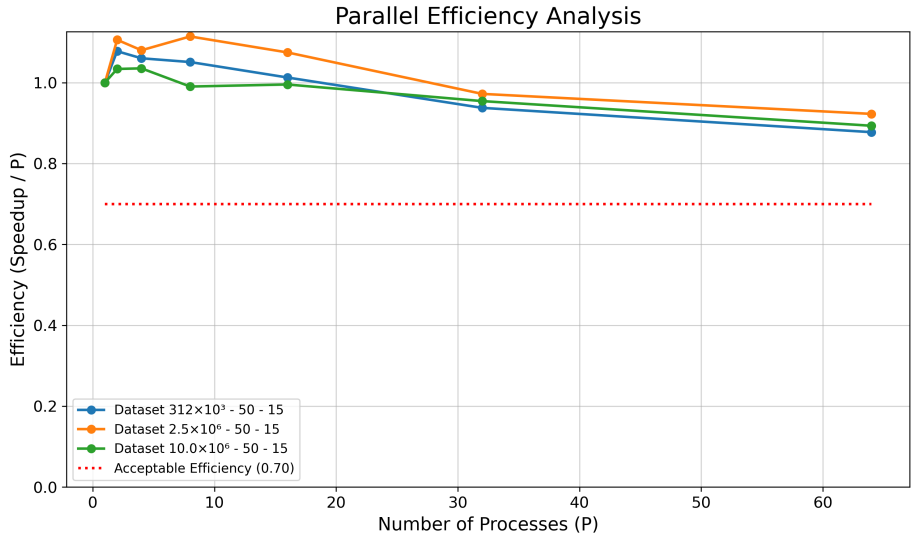

Figure 6: Efficiency plot for different dataset sizes (small, medium, large). The dashed line indicates the target efficiency of 0.70.

Analyzing the results from Tables 2 and 3, we observe that as the dataset size increases, the speedup approaches linearity, and the efficiency remains relatively high even when the number of processes increases. This indicates that our parallel implementation makes effective use of additional computational resources as the workload grows.
Examining the main diagonal of the table, where the problem size grows proportionally to the number of processes, we find that the efficiency remains almost constant; our implementation is therefore *weakly scalable*.
Conversely, when examining each column of the table, corresponding to a fixed problem size, we observe a gradual decline as the number of processes increases. This is expected, as the communication overhead becomes more significant when the process count increases. Consequently, our implementation does not exhibit perfect *strong scalability*.

NP	0.156	0.312	0.625	1.25	2.5	5	10
1	1.000	1.000	1.000	1.000	1.000	1.000	1.000
2	1.003	1.084	0.898	1.045	1.106	1.042	1.035
4	0.996	1.070	0.918	1.016	1.081	1.041	1.033
8	1.028	1.081	0.910	0.967	1.142	1.050	1.039
16	0.921	1.031	0.943	0.969	1.093	0.949	1.028
32	0.956	0.962	0.830	0.910	1.008	0.963	0.981
64	0.883	0.925	0.821	0.849	0.961	0.944	0.923

Table 4: E-Step efficiency for different dataset sizes and number of processes.

NP	0.156	0.312	0.625	1.25	2.5	5	10
1	1.000	1.000	1.000	1.000	1.000	1.000	1.000
2	0.963	0.993	0.875	1.035	1.107	0.995	1.022
4	0.857	0.924	0.773	0.947	1.064	0.949	1.072
8	0.866	0.738	0.557	0.691	0.826	0.549	0.601
16	0.576	0.793	0.745	0.685	0.866	0.800	0.690
32	0.673	0.673	0.441	0.594	0.644	0.701	0.695
64	0.543	0.490	0.398	0.437	0.584	0.568	0.617

Table 5: M-Step efficiency for different dataset sizes and number of processes.

Analyzing the results presented in tables 4 and 5, we observe that, as expected,

the E-step maintains consistent high efficiency across all tested configurations. In contrast, the M-step exhibits a noticeable decline in efficiency as the number of processes increases.

Other datasets To further investigate this behavior, we conducted additional experiments on datasets with varying characteristics. Specifically, we increased the number of features ($D = 300$) and the number of clusters ($K = 20$). The results are summarized in table 8.

N_Processes	0.033M	0.066M	0.131M	0.262M	0.524M	1.049M	2.097M
1	1.000	1.000	1.000	1.000	1.000	1.000	1.000
2	1.038	1.063	0.944	1.086	1.000	1.056	0.832
4	1.046	1.121	1.022	0.983	0.953	1.041	0.867
8	1.036	1.030	0.989	1.019	0.946	1.013	0.879
16	0.899	0.954	0.982	0.985	0.874	0.960	0.842
32	0.880	0.989	0.929	0.943	0.838	0.887	0.787
64	0.797	0.865	0.874	0.879	0.830	0.918	0.757

Table 6: Efficiency (Speedup / p) for different dataset sizes and number of processes.

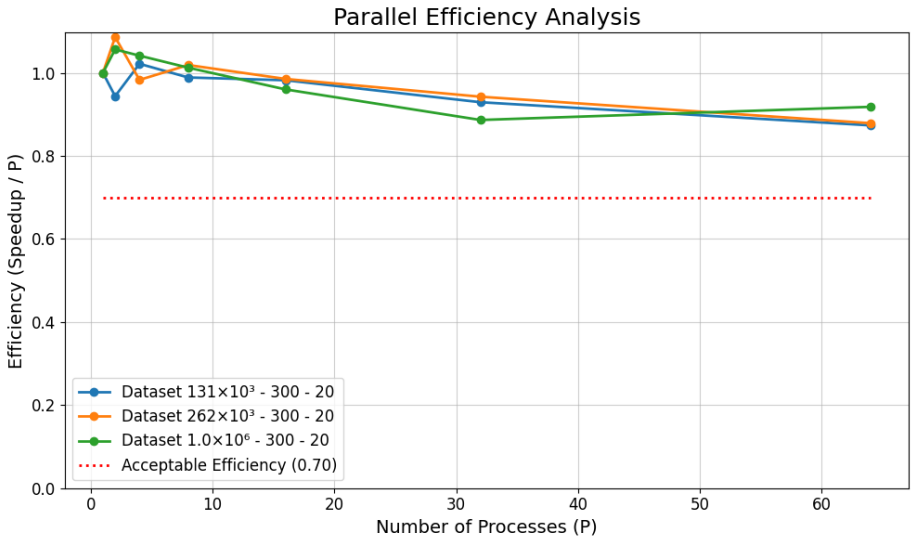

Figure 7: Efficiency plot for different dataset sizes (small, medium, large) with increased features and clusters. The dashed line indicates the target efficiency of 0.70.

N_Processes	0.033M	0.066M	0.131M	0.262M	0.524M	1.049M	2.097M
1	1.000	1.000	1.000	1.000	1.000	1.000	1.000
2	1.026	1.074	0.694	1.037	1.024	1.021	0.820
4	1.010	0.901	0.817	0.632	0.633	1.002	0.752
8	0.749	0.636	0.672	0.840	0.662	0.674	0.589
16	0.523	0.510	0.806	0.693	0.555	0.767	0.788
32	0.621	0.581	0.538	0.654	0.540	0.415	0.490
64	0.322	0.324	0.468	0.452	0.408	0.569	0.292

Table 7: Efficiency for M-Step for different dataset sizes and number of processes.

The findings confirm our initial observation. The E-step remains efficient across different configurations, whereas the M-step efficiency deteriorates due to increased communication costs associated with larger feature dimensions and more clusters.

4.2 Hybrid Approach Results

For the hybrid approach experiments, we explored different combinations of nodes, CPUs per node, and threads per process, as listed in Table 4.2. *Nodes* represents the number of computing nodes used, *Rank/Node* indicates the number of MPI processes per node, *Threads/Rank* is the number of threads allocated to each MPI process, and *Threads* is the total number of threads used in the configuration. By matching the threads to the number of CPUs, we can

Nodes	Rank/Node	Threads/Rank	Threads
1	1	1	1
1	1	2	2
1	1	4	4
1	1	8	8
2	1	8	16
2	2	8	32
4	2	8	64

compare the hybrid approach directly to the pure MPI implementation. For full results of the Hybrid approach, consult the appendix.

N_Processes	0.033M	0.066M	0.131M	0.262M	0.524M	1.049M	2.097M
1	1.000	1.000	1.000	1.000	1.000	1.000	1.000
2	1.040	0.975	0.984	0.985	0.968	1.029	0.840
4	1.075	1.057	0.970	1.068	0.955	1.052	0.929
8	1.055	0.981	0.978	1.000	0.996	1.038	0.907
16	1.003	0.929	0.880	0.893	0.915	0.923	0.860
32	0.974	0.872	0.828	0.970	0.888	0.903	0.822
64	0.925	0.862	0.854	0.922	0.912	0.897	0.868

Table 8: Efficiency for Hybrid approach for different dataset sizes and number of processes.

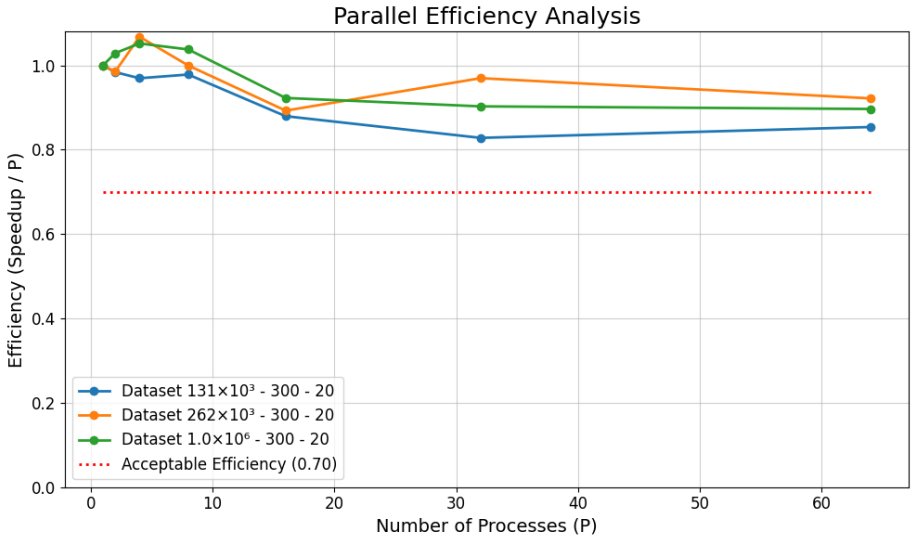

Figure 8: Hybrid Efficiency plot for different dataset sizes (small, medium, large). The dashed line indicates the target efficiency of 0.70.

N_Processes	0.033M	0.066M	0.131M	0.262M	0.524M	1.049M	2.097M
1	1.000	1.000	1.000	1.000	1.000	1.000	1.000
2	1.041	0.941	0.978	1.046	0.988	1.109	0.761
4	1.092	1.052	0.965	1.085	0.917	1.083	0.842
8	1.053	0.976	0.968	0.988	1.044	1.072	0.764
16	0.843	0.828	0.634	0.519	0.850	0.747	0.702
32	0.812	0.575	0.521	0.703	0.704	0.772	0.566
64	0.620	0.501	0.707	0.565	0.743	0.694	0.600

Table 9: Efficiency for Hybrid M-Step for different dataset sizes and number of processes.

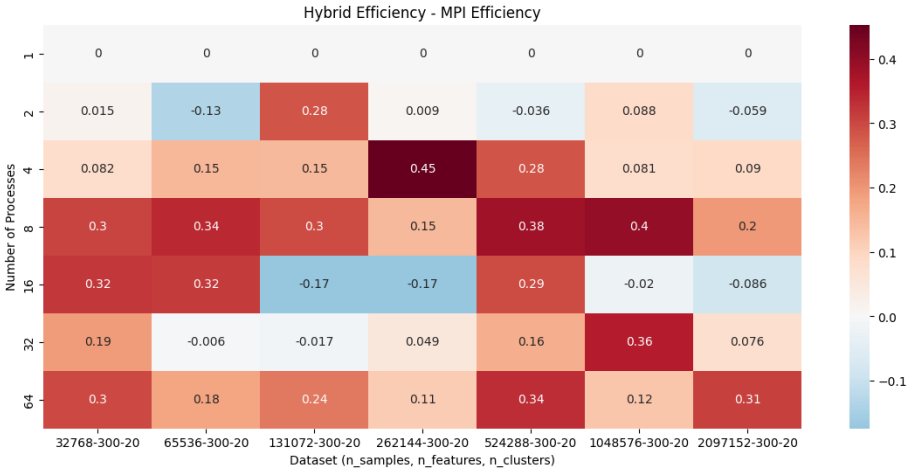

Figure 9: M-Step Efficiency Heatmap for difference between Pure MPI and Hybrid approach. Red indicates higher efficiency in Hybrid, blue indicates higher efficiency in Pure MPI.

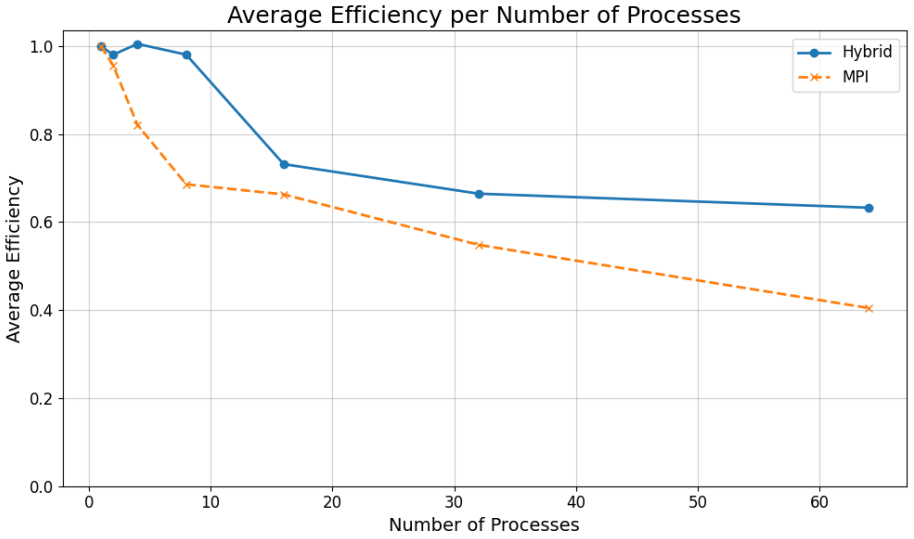

Figure 10: Average M-Step Efficiency Comparison between Pure MPI and Hybrid approach.

Final Evaluation From the results obtained, it is evident that both the pure MPI and hybrid approaches exhibit excellent efficiency, demonstrating good weak scaling properties and obtaining efficiencies above 0.70 in most configura-

tions.
The pure MPI implementation demonstrates strong scalability up to a point, but once the communication overhead becomes significant, its efficiency starts to drop.
The hybrid approach, on the other hand, manages to maintain a more consistent efficiency across different configurations. This is particularly noticeable in the M-step, where the hybrid model leverages shared memory parallelism effectively, allowing it to significantly outperform the pure MPI implementation in several scenarios.
Unfortunately, we were unable to directly compare our implementation against other existing solutions on the same hardware architecture. However, the performance metrics obtained are consistent with the state-of-the-art literature regarding parallel implementations of the Expectation-Maximization (EM) algorithm.

References

[1] D. Reynolds, *Gaussian Mixture Models*. Boston, MA: Springer US, 2009.

[2] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

[3] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the em algorithm," *Journal of the Royal Statistical Society: Series B*, vol. 39, no. 1, pp. 1–38, 1977.

[4] A. Kak, "Expectation–maximization algorithm for clustering multidimensional numerical data: An rvl tutorial," Tech. Rep. —, Purdue University, March 3 2024.

[5] M. C. Altinigneli, C. Plant, and C. Böhm, "Massively parallel expectation maximization using graphics processing units," in *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '13, (New York, NY, USA), p. 838–846, Association for Computing Machinery, 2013.

[6] S. X. Lee, K. L. Leemaqz, and G. J. McLachlan, "A simple parallel em algorithm for statistical learning via mixture models," in *2016 International Conference on Digital Image Computing: Techniques and Applications (DICTA)*, pp. 1–8, 2016.

[7] I. Azizi, "Parallelization in python - an expectation-maximization application," tech. rep., Université de Lausanne, Département des Opérations (DO), June 2023. Inspired by CUSO Informatique 2023 Winter School.

A Appendix

Pseudocode

Algorithm 4 E-step: Computation of Responsibilities

Require: Dataset $X \in \mathbb{R}^{N \times D}$, parameters μ_k, σ_k, π_k for $k = 1, \ldots, K$
Ensure: Responsibilities γ_{ik}

for $i = 1$ to N **do**
    denom $\leftarrow 0$
    for $k = 1$ to K **do**
        $\gamma_{ik} \leftarrow \pi_k \cdot \mathcal{N}(X[i] \mid \mu_k, \Sigma_k)$ ▷ Gaussian density using 6
        denom $\leftarrow$ denom $+ \gamma_{ik}$
    end for
    for $k = 1$ to K **do**
        $\gamma_{ik} \leftarrow \gamma_{ik}/$denom
    end for
end for

Algorithm 5 M-step: Parameter Update

Require: Dataset $X \in \mathbb{R}^{N \times D}$, responsibilities γ_{ik}
Ensure: Updated parameters μ_k, σ_k, π_k

Initialize accumulators: $N_k \leftarrow 0$, $\mu_{kd}^{\text{acc}} \leftarrow 0$, $\sigma_{kd}^{\text{acc}} \leftarrow 0$
for $i = 1$ to N **do**
    for $k = 1$ to K **do**
        $N_k \leftarrow N_k + \gamma_{ik}$
        for $d = 1$ to D **do**
            $\mu_{kd}^{\text{acc}} \leftarrow \mu_{kd}^{\text{acc}} + \gamma_{ik} \cdot X[i, d]$
        end for
    end for
end for
for $k = 1$ to K **do**
    for $d = 1$ to D **do**
        $\mu_{kd} \leftarrow \mu_{kd}^{\text{acc}} / N_k$
    end for
end for
for $i = 1$ to N **do**
    for $k = 1$ to K **do**
        for $d = 1$ to D **do**
            $\sigma_{kd}^{\text{acc}} \leftarrow \sigma_{kd}^{\text{acc}} + \gamma_{ik} \cdot (X[i, d] - \mu_{kd})^2$
        end for
    end for
end for
for $k = 1$ to K **do**
    for $d = 1$ to D **do**
        $\sigma_{kd} \leftarrow \sigma_{kd}^{\text{acc}} / N_k$
    end for
    $\pi_k \leftarrow N_k / N$
end for

Algorithm 7 Gaussian Density Computation with Diagonal Covariance

Require: $x \in \mathbb{R}^D$, $\mu \in \mathbb{R}^D$, $\sigma \in \mathbb{R}^D$
Ensure: $\mathcal{N}(x \mid \mu, \sigma)$
logdet $\leftarrow 0$
quad $\leftarrow 0$
for $d = 1$ to D **do**
    $var \leftarrow \sigma[d]$
    logdet $\leftarrow$ logdet $+ \log(var)$
    diff $\leftarrow x[d] - \mu[d]$
    quad $\leftarrow$ quad $+ \frac{\text{diff}^2}{var}$
end for
exponent $\leftarrow -0.5 \cdot (D \cdot \log 2\pi +$ logdet $+$ quad$)$
return exp(exponent)

Algorithm 8 Clustering: Label Assignment from Responsibilities

Require: Responsibilities matrix $\gamma \in \mathbb{R}^{N \times K}$
Ensure: Predicted labels $\hat{y} \in \{1, \ldots, K\}^N$
for $i = 1$ to N **do**
    max_resp $\leftarrow \gamma_{i1}$ ▷ Initialize with first cluster responsibility
    $\hat{y}_i \leftarrow 1$
    for $k = 2$ to K **do**
        if $\gamma_{ik} >$ max_resp **then**
            max_resp $\leftarrow \gamma_{ik}$
            $\hat{y}_i \leftarrow k$
        end if
    end for
end for

Other Datasets Full Results

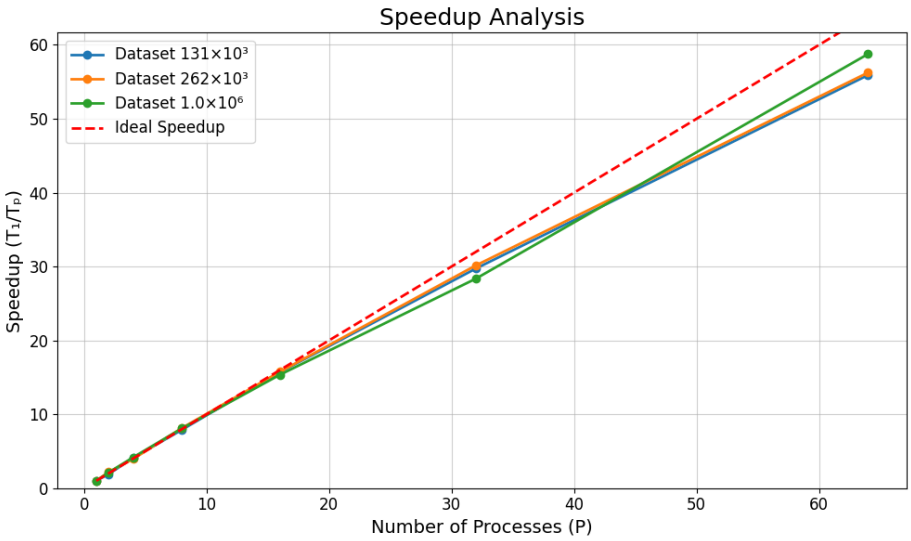

Figure 11: Speedup plot for different dataset sizes (small, medium, large) with increased features and clusters. The dashed line represents the ideal linear speedup.

Hybrid Full Results

Here are the complete results for the Hybrid approach, including execution time, speedup, and E-Step efficiency.

N_Processes	0.033M	0.066M	0.131M	0.262M	0.524M	1.049M	2.097M
1	633.233	1177.423	2191.697	4637.819	8652.474	17008.507	30070.722
2	304.441	603.905	1113.554	2354.183	4471.163	8265.101	17905.912
4	147.311	278.408	565.111	1085.605	2265.491	4041.279	8094.882
8	75.020	149.955	280.005	579.793	1085.627	2048.421	4145.526
16	39.447	79.223	155.722	324.658	590.897	1151.906	2186.361
32	20.313	42.183	82.698	149.456	304.556	588.653	1143.465
64	10.694	21.348	40.104	78.604	148.252	296.361	541.036

Table 10: Execution Time (seconds) for different dataset sizes and number of processes.

N_Processes	0.033M	0.066M	0.131M	0.262M	0.524M	1.049M	2.097M
1	1.000	1.000	1.000	1.000	1.000	1.000	1.000
2	2.080	1.950	1.968	1.970	1.935	2.058	1.679
4	4.299	4.229	3.878	4.272	3.819	4.209	3.715
8	8.441	7.852	7.827	7.999	7.970	8.303	7.254
16	16.053	14.862	14.074	14.285	14.643	14.766	13.754
32	31.173	27.912	26.503	31.031	28.410	28.894	26.298
64	59.214	55.155	54.650	59.002	58.363	57.391	55.580

Table 11: Speedup (T1/Tp) for different dataset sizes and number of processes.

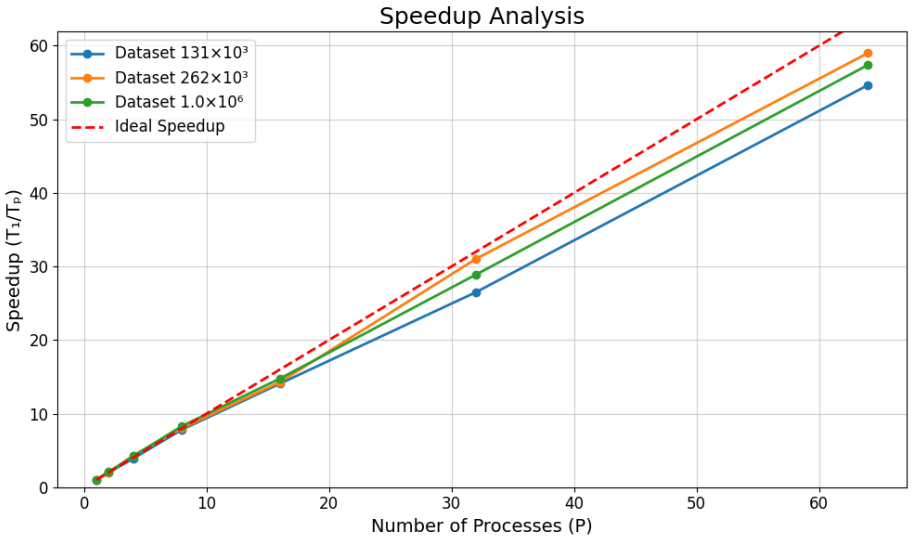

Figure 12: Hybrid Speedup plot for different dataset sizes (small, medium, large). The dashed line indicates the ideal linear speedup.

N_Processes	0.033M	0.066M	0.131M	0.262M	0.524M	1.049M	2.097M
1	1.000	1.000	1.000	1.000	1.000	1.000	1.000
2	1.040	0.977	0.984	0.982	0.966	1.024	0.845
4	1.074	1.058	0.970	1.067	0.957	1.050	0.935
8	1.055	0.982	0.979	1.001	0.994	1.036	0.917
16	1.013	0.935	0.898	0.929	0.919	0.937	0.872
32	0.984	0.896	0.854	0.990	0.902	0.913	0.845
64	0.948	0.895	0.863	0.955	0.924	0.914	0.893

Table 12: E-Step efficiency for different dataset sizes and number of processes.